

ILP2 : Boucles, affectations, fonctions

MU4IN501 – DLP : Développement d'un langage de programmation
Master STL, Sorbonne Université

Antoine Miné

Cours 5

10 octobre 2023

Année 2023–2024



Développement d'un **langage** de programmation :
interprète et **compilateur**.

Par étapes, avec ajout progressif de fonctionnalités.

- **ILP1** : langage de base

- constantes (entiers, flottants, booléens, chaînes),

- opérateurs (+, −, ×, ...),

- appels de primitives (print, ...),

- blocs locaux (let x = ... in ...),

- alternatives (if ... then ... else)

- **ILP2** : ajout des boucles, des affectations, des variables et fonctions globales

- **ILP3** : ajout des exceptions et fonctions de première classe

- **ILP4** : ajout des classes et des objets

Passage à ILP2 !

En TME, un nouveau projet GitLab nommé ILP2 contient ILP1+ILP2,
il faudra en faire un *fork* sur le serveur GitLab, puis un *clone* local dans Eclipse...

- Traits de langage ajoutés : variables globales, affectations, boucles.
- Extension de la grammaire et de l'analyse syntaxique.
- Interprétation et compilation des boucles.
- Interprétation et compilation des fonctions et de l'affectation.

Détermination des variables globales.

Accès aux variables, appels de fonctions.

- Voir les étapes pour **étendre** ILP. (utile en TME et pour l'examen !)
- Revoir l'analyse syntaxique et ANTLR 4.
- Revoir la portée des variables et la gestion des environnements.
- Revoir l'**interprète**.
- Aspects statiques et dynamiques et **analyse statique**.
- Revoir la génération de code C et les **schémas de compilation**.

Affectations

```
let x = 10 in (  
  print(x);  
  x = x + 1;  
  print(x)  
)
```

Variables globales

```
g = 1;  
(let x = 10 in g = g + x);  
print(g)
```

Retour d'affectation

```
let x = 10 in  
g = (x = x + 1);  
print(g)
```

Erreur d'initialisation

```
print(g);  
g = 12
```

Boucles

```
(let x = 0 in
  while x < 10 do (
    g = x;
    x = x + 1
  )
);
print(g)
```

递归函数

Fonctions

```
function myprint(x)
(
    print("*** ");
    print(x);
    newline()
);

myprint(12);
myprint("help!")
```

Fonctions récursives

```
function fact(n)
(
    if n > 1
    then n * fact(n - 1)
    else 1
);

fact(4)
```

Variables globales et fonctions

```
function f()
    glob = 99;

f();
print(glob)
```

Récursion mutuelle

```
function odd(n)
(
  if n == 0 then false
  else if n == 1 then true
  else even (n - 1)
);

function even(n)
(
  if n == 0 then true
  else if n == 1 then false
  else odd (n - 1)
);

odd(56)
```

Fonctions en argument

```
function deuxfois(x)
  2 * x;

function twice(f, x)
  f(f(x));

twice(deuxfois, 1500) - 6
```

Fonctions comme valeur

```
function deuxfois(x)
  2 * x;

let f = deuxfois in
f (if 1 < 2 then 3000 else 10) - 8
```


Boucles :

- en ILP : `while c do b`
- en C et en Java :
 - `while (c) b`
 - `do b while (c)`
 - `for (a;b;c) d`
 - et aussi : `break`, `continue`
- en Python :
 - `while c: b`
 - `for i in l: b`

Déclaration et portée des variables :

- en ILP :
 - uniquement les locales sont explicitement déclarées (argument de fonction ou bloc `let`)
 - **globales non déclarées**
tout ce qui n'est pas explicitement local est global
 - une variable locale est forcément initialisée,
une variable globale peut être lue avant d'être initialisée ! (**erreur à l'exécution**)
- en C :
 - **tout** doit être déclaré
 - utilisation de mots-clés pour distinguer la portée : `extern` (et `auto`)
 - pas d'initialisation automatique (hors `static`)
- en Python : **aucune déclaration**
 - un bloc syntaxique par fonction (`if`, `while`, etc. n'introduisent pas de bloc de portée!)
 - toute variable affectée dans une fonction est considérée comme locale
 - toute variable utilisée et non affectée est considérée comme globale
 - mots-clés pour altérer la portée : `global`, `nonlocal`
 - il est extrêmement facile de se tromper !
⇒ utilisation de la mauvaise variable, d'une variable non définie

Comparaison (3/3) : les fonctions

Déclaration et appel de fonctions :

- en ILP : **deux passes** (y compris lors de l'interprétation)
 - lecture du code complet pour extraire les définitions
 - exécution du code avec les références résolues
- en C : **une seule passe**
 - une définition ne peut référencer que des fonctions définies avant
 - ou déclarées avant avec un prototype

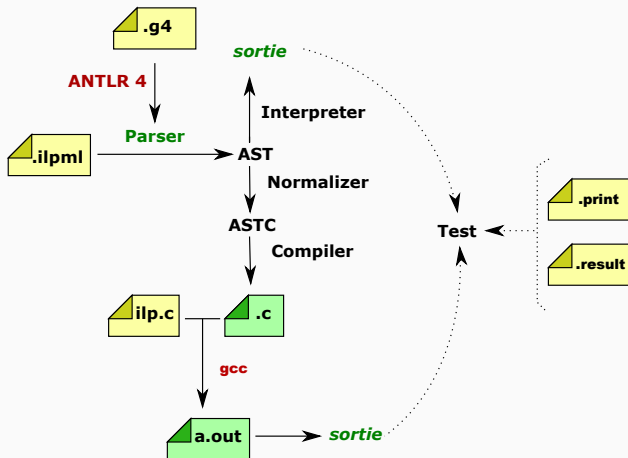
```
void even(int);  
void odd(int x) { ...even(...) ...}  
void even(int x) { ...odd (...) ...}
```
- en Python : **une seule passe**
 - exécuter une déclaration enregistre le corps de la fonction qui peut contenir des identificateurs non définis
 - résolution des références au moment de l'appel de la fonction

Fonctions comme valeurs :

- en ILP et en Python : une **fonction** est une **valeur**
- en C : pointeur sur une fonction `int (*p)(float) = &f`
puis appel indirect par pointeur `p(1.2)`, ou `(*p)(1.2)`

ILP3 ajoutera les **fonction locales**, présentes en Python mais absentes en C.

Schéma général d'ILP (rappel)



Étapes d'une extension (utile en TME)

Extension de la syntaxe :

- écrire une grammaire ANTLR 4 ;
- ajouter des nœuds `IAST`, `AST` et `IASTC`, `ASTC` ;
- écrire un *Listener* obéissant à l'interface produite par ANTLR.

Extension de la base de tests (`.ilpml`, `.result`, `.print`).

Extension des visiteurs : `IAST(C)visitor` :

- classes `Interpreter`, `Normalizer`, `Compiler`.

Extension des primitives et opérateurs de l'interprète.

Extension de la bibliothèque d'exécution C :

- type `ILP_Object` dans `ilp.h` ;
- primitives dans `ilp.c`.

Extension des classes de test `InterpreterTest` et `CompilerTest`.

Ces étapes ne sont pas toutes nécessaires pour chaque extension !

Règles de programmation pour les extensions

En Java :

- **pas de modification du code existant** ;
- **nouvelles classes** dans des **packages séparés** `com.paracamplus.ilp2...` ;
- réutilisation par **héritage** ;
- **motifs** visiteur, composite et fabrique facilitant l'extensibilité.

En ANTLR 4 :

- difficile d'hériter d'une grammaire `.g4` pour y ajouter des règles ;
- si la grammaire change, la classe *Listener* ne peut pas être réutilisée ;
(ANTLR génère une nouvelle interface *Listener* sans lien d'héritage avec l'ancienne)

⇒ **copie nécessaire**, puis modification de la grammaire et du *Listener*.

En C :

- `ilp.c` et `ilp.h` implantent déjà tout ILP1 à ILP4...
- difficile d'étendre un type `struct` ⇒ **autorisation de modifier `ilp.h`** ;
- déclarer et définir les fonctions **dans des `.c` et `.h` séparés**.

Hierarchie des *packages* pour ILP2

- analyse syntaxique et AST :
 - `com.paracamplus.ilp2.interfaces`
 - `com.paracamplus.ilp2.ast`
 - `com.paracamplus.ilp2.parser.ilpml`
- interprète :
 - `com.paracamplus.ilp2.interpreter`
 - `com.paracamplus.ilp2.interpreter.test`
- compilateur :
 - `com.paracamplus.ilp2.compiler`
 - `com.paracamplus.ilp2.compiler.interfaces`
 - `com.paracamplus.ilp2.compiler.ast`
 - `com.paracamplus.ilp2.compiler.normalizer`
 - `com.paracamplus.ilp2.compiler.test`
- voir aussi : [SamplesILP2/](#) et [ANTLRGrammars/ILPMLgrammar2.g4](#)
mais pas de modification de la bibliothèque d'exécution C/!

Chaque *package* ILP2 contient :

- des **nouvelles** classes et interfaces ;
- des classes et interfaces qui **héritent** des classes et interfaces **de même nom** dans le *package* ILP1 correspondant.

Classes et interfaces de ILP2 dans com.paracampus.ilp2...

... .interfaces

IASTassignment
IASTfunctionDefinition
IASTloop
IASTdeclaraton
IASTprogram
IASTfactory
IASTvisitor

... .ast

ASTassignment
ASTfunctionDefinition
ASTloop
ASTprogram
ASTfactory

... .parser.ilpml

ILPMLlistener
ILPMLParser
ILPMLProgramPrinter

... .test

WholeTestSuite

... .interpreter

Interpreter

... .interpreter.test

InterpreterTest

... .compiler.interfaces

IASTCfunctionDefinition
IASTCprogram
IASTCvisitor

... .compiler.ast

ASTCfunctionDefinition
ASTCglobalFunctionVariable
ASTCglobalInvocation
ASTCprogram

... .compiler.normalizer

INormalizationFactory
NormalizatioFactory
Normalizer

... .compiler

FreeVariableCollector
GlobalVariableCollector
Compiler

... .compiler.test

CompilerTest

Les boucles

ILPMLgrammar2.g4

```
grammar ILPMLgrammar2;  
...  
expr returns [com.paracampus.ilp1.interfaces.IASTexpression node]  
    : ...  
    | ...  
    | 'while' condition=expr 'do' body=expr # Loop  
    ;
```

Boucle *while* avec :

- une expression pour la **condition**;
- une expression pour le corps : **body**.

Une boucle est aussi une **expression** ! elle retourne **false**

La règle est nommée **Loop**.

Elle est en fin de fichier $\implies$ elle a une très faible priorité.

`while true do 1 + 1` est lu comme `while true do (1 + 1)`

IASTloop.java

```
package com.paracamplus.ilp2.interfaces;

public interface IASTloop extends IASTexpression
{
    IASTexpression getCondition ();
    IASTexpression getBody ();
}
```

L'interface suit précisément la syntaxe concrète.

```
package com.paracamplus.ilp2.ast;
import com.paracamplus.ilp2.interfaces.IASTvisitor;
import ...

public class ASTloop extends ASTexpression implements IASTloop, IASTvisitable
{
    private final IASTexpression condition;
    private final IASTexpression body;

    public ASTloop (IASTexpression condition, IASTexpression body)
    { this.condition = condition; this.body = body; }

    @Override public IASTexpression getCondition() { return condition; }
    @Override public IASTexpression getBody() { return body; }

    @Override public <Result, Data, Anomaly extends Throwable> Result accept(
        com.paracamplus.ilp1.interfaces.IASTvisitor<...> visitor, Data data)
        throws Anomaly
    { return ((IASTvisitor <Result, Data, Anomaly>) visitor).visit(this, data); }
}
```

- conteneur d'attributs **condition** et **body** en lecture seule ;
- **visiteur ILP2** pour l'orientation (*dispatch*)
prenant un visiteur ILP1 en argument pour la **compatibilité**. (détails un peu plus loin...)

IASTfactory.java (ILP2)

```
public interface IASTfactory
extends com.paracampus.ilp1.interfaces.IASTfactory
{
    IASTexpression newLoop(IASTexpression condition, IASTexpression body);
    ...
}
```

ASTfactory.java (ILP2)

```
public class ASTfactory
extends com.paracampus.ilp1.ast.ASTfactory
implements IASTfactory
{
    @Override
    public IASTloop newLoop(IASTexpression condition, IASTexpression body) {
        return new ASTloop(condition, body);
    }
    ...
}
```

Utilisation systématique de la fabrique pour créer des objets obéissant aux interfaces des nœud **IAST...** en restant indépendant de la hiérarchie de classes.

Ne pas utiliser **new AST...(...)** directement !

Extension de l'analyse syntaxique : *Listener*

ILPMLListener.java (ILP2)

```
public class ILPMLListener implements ILPMLgrammar2Listener {
    protected IASTfactory factory;

    public ILPMLListener(IASTfactory factory)
    { super(); this.factory = factory; }
    ...

    @Override public void enterLoop(LoopContext ctx) { }

    @Override public void exitLoop(LoopContext ctx) {
        ctx.node = factory.newLoop(ctx.condition.node, ctx.body.node);
    }
}
```

- ANTLR génère une interface `ILPMLgrammar2Listener`
mais elle n'hérite pas de `ILPMLgrammar1Listener`
les types `Context` d'`ILPMLgrammar1Parser` ne sont équivalents à ceux correspondant dans `ILPMLgrammar2Parser`
⇒ impossible d'hériter des méthodes `enter` et `exit` du listener d'ILP1 !
- `ILPMLListener` de ILP2 réimplante toutes les méthodes de ILP1
par recopie !
- `ILPMLListener` utilise la fabrique ILP2
- `ILPMLListener` ajoute les méthodes `exit` pour les nouvelles règles

IASTvisitor.java (rappel ILP1)

```
public interface IASTvisitor<R, D, A extends Throwable> {  
    R visit(IASTalternative iast, D data) throws A;  
    R visit(IASTbinaryOperation iast, D data) throws A;  
    R visit(IASTblock iast, D data) throws A;  
    R visit(IASTboolean iast, D data) throws A;  
    R visit(IASTfloat iast, D data) throws A;  
    R visit(IASTinteger iast, D data) throws A;  
    R visit(IASTinvocation iast, D data) throws A;  
    R visit(IASTsequence iast, D data) throws A;  
    R visit(IASTstring iast, D data) throws A;  
    R visit(IAStunaryOperation iast, D data) throws A;  
    R visit(IASTvariable iast, D data) throws A;  
}
```

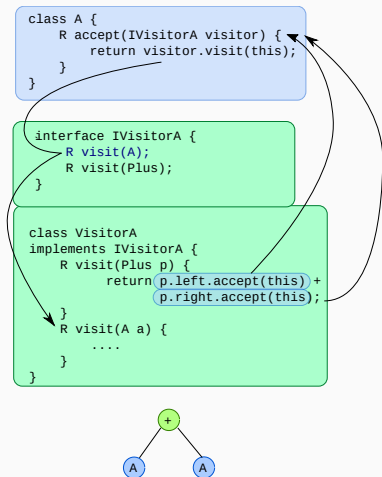
IASTvisitor.java (version ILP2)

```
public interface IASTvisitor<R, D, A extends Throwable>  
    extends com.paracampus.ilp1.interfaces.IASTvisitor<R, D, A> {  
    R visit(IASTassignment iast, D data) throws A;  
    R visit(IASTloop iast, D data) throws A;  
}
```

IASTvisitable.java (tous ILP)

```
public interface IASTvisitable {  
    <R, D, A extends T> R accept(IASTvisitor<R, Data, A> visitor, D data) throws A;  
}
```

Visiteurs : principe



Visiteurs : extension

```
class A {  
    R accept(IVisitorA visitor) {  
        return visitor.visit(this);  
    }  
}
```

```
interface IVisitorA {  
    R visit(A);  
    R visit(Plus);  
}
```

```
class VisitorA  
implements IVisitorA {  
    R visit(Plus p) {  
        return p.left.accept(this) +  
               p.right.accept(this);  
    }  
    R visit(A a) {  
        ....  
    }  
}
```

```
class B {  
    R accept(IVisitorA visitor) {  
        return ((IVisitorAB)visitor).visit(this);  
    }  
}
```

```
interface IVisitorAB  
extends IVisitorA {  
    R visit(B);  
}
```

```
class VisitorAB  
extends VisitorA  
implements IVisitorAB {  
    R visit(B b) {  
        ....  
    }  
}
```



```
public class Interpreter extends com.paracamplus.ilp1.interpreter.Interpreter
implements IASTvisitor<Object, ILexicalEnvironment, EvaluationException> {

    public Interpreter(IGlobalVariableEnvironment globalVariableEnvironment,
                      IOperatorEnvironment operatorEnvironment)
    { super(globalVariableEnvironment, operatorEnvironment); }

    @Override public Object visit(IASTloop iast, ILexicalEnvironment lexenv) throws EvaluationException
    {
        while ( true ) {
            Object condition = iast.getCondition().accept(this, lexenv);
            if ( condition instanceof Boolean ) {
                Boolean c = (Boolean) condition;
                if ( ! c ) break;
            }
            iast.getBody().accept(this, lexenv);
        }
        return Boolean.FALSE;
    }
    ...
}
```

- héritage de l'interprète ILP1 et réutilisation de son constructeur ;
- ajout du traitement des boucles : exécuter le corps, tant que la condition ne s'évalue pas en le booléen `false`.

Configuration et exécution du nouvel interprète

InterpreterTest.java (ILP2)

```
@RunWith(Parameterized.class) public class InterpreterTest
extends com.paracampus.ilp1.interpreter.test.InterpreterTest {

    public InterpreterTest(final File file) { super(file); }

    public void configureRunner(InterpreterRunner run) throws EvaluationException {
        IASTfactory factory = new ASTfactory();
        run.setILPMLParser(new ILPMLParser(factory));

        StringWriter stdout = new StringWriter();
        run.setStdout(stdout);
        IGlobalVariableEnvironment gve = new GlobalVariableEnvironment();
        GlobalVariableStuff.fillGlobalVariables(gve, stdout);
        IOperatorEnvironment oe = new OperatorEnvironment();
        OperatorStuff.fillUnaryOperators(oe);
        OperatorStuff.fillBinaryOperators(oe);
        Interpreter interpreter = new Interpreter(gve, oe);
        run.setInterpreter(interpreter);
    }
}
```

- héritage de la version ILP1 ;
- la configuration de l'interprète `configureRunner` est changée ;
le code semble le même que la version ILP1,
mais la version ILP2 de `ASTfactory`, `ILPMLParser` et `Interpreter` est utilisée !
(c.f. : directives `import com.paracampus.ilp2....`)

Normalizer.java (ILP2)

```
public class Normalizer
extends com.paracampus.ilp1.compiler.normalizer.Normalizer
implements IASTvisitor<IASTexpression, INormalizationEnvironment, CompilationException> {

    public Normalizer (INormalizationFactory factory)
    { super(factory); }

    @Override public IASTexpression visit(IASTloop iast, INormalizationEnvironment env)
    throws CompilationException {
        IASTexpression newcondition = iast.getCondition().accept(this, env);
        IASTexpression newbody = iast.getBody().accept(this, env);
        return ((INormalizationFactory)factory).newLoop(newcondition, newbody);
    }
}
```

- pas de nœud ASTC spécifique pour la boucle : on réutilise **ASTloop** ;
la boucle n'introduit pas de nouvelle variable. . .
- néanmoins, le visiteur de **ASTloop** doit :
 - s'appeler récursivement sur ses arguments **condition** et **body** ;
 - reconstruire le nœud **ASTloop** avec la fabrique.

Schéma de compilation

$\xrightarrow{\quad}^d$
`while cond do body`

```
while (1)
{
    ILP_Object tmp;
     $\xrightarrow{\quad}^d$   
cond(tmp==)
    if (ILP_isEquivalentToTrue(tmp)) {
         $\xrightarrow{\quad}^d$   
body(void)
    } else {
        break;
    }
}
 $\xrightarrow{\quad}^d$   
false
```

Compiler.java (ILP2)

```
@Override public Void visit(IASTloop iast, Context context)
throws CompilationException
{
    emit("while ( 1 ) { \n");
    IASTvariable tmp = context.newTemporaryVariable();
    emit("  ILP_Object " + tmp.getMangledName() + "; \n");
    Context c = context.redirect(new AssignDestination(tmp));
    iast.getCondition().accept(this, c);
    emit("  if ( ILP_isEquivalentToTrue(");
    emit(tmp.getMangledName());
    emit(") ) { \n");
    Context cb = context.redirect(VoidDestination.VOID_DESTINATION);
    iast.getBody().accept(this, cb);
    emit("\n} else { \n");
    emit("    break; \n");
    emit("\n}\n\n");
    whatever.accept(this, context);
    return null;
}
```

- ILP1 définit : `IASTboolean whatever = new ASTboolean("false");`

Variables globales, fonctions, affectations

```
prog returns [com.paracampus.ilp2.interfaces.IASTprogram node]
: (defs+=globalFunDef ';'?)* (exprs+=expr ';'?) * EOF
;

globalFunDef returns [com.paracampus.ilp2.interfaces.IASTfunctionDefinition node]
: 'function' name=IDENT '(' vars+=IDENT? (',' vars+=IDENT)* ')' body=expr
;

expr returns [com.paracampus.ilp1.interfaces.IASTexpression node]
| ...
| var=IDENT '=' val=expr # VariableAssign
| ...
```

- la règle des **programmes** est changée :
suite de **déclarations de fonctions**, suivies d'expressions ;
- une fonction contient :
 - le mot-clé **function** ;
 - le nom de la fonction **name** ;
 - une liste **vars** de nom de variables, séparées par **,** et entre parenthèses ;
 - le corps de la fonction **body**, qui est une expression ;

⇒ règle **globalFunDef**, de type **IASTfunctionDefinition**
- la règle d'affectation est simplement **var = val**, c'est une expression.

Interfaces de nœuds de l'AST

IASTdeclaration.java

```
public interface IASTdeclaration extends IAST { }
```

IASTfunctionDefinition.java

```
public interface IASTfunctionDefinition extends IASTdeclaration, Inamed {  
    IASTvariable getFunctionVariable();  
    IASTvariable[] getVariables();  
    IASTexpression getBody();  
}
```

IASTassignment.java

```
public interface IASTassignment extends IASTexpression {  
    IASTvariable getVariable();  
    IASTexpression getExpression();  
}
```

- interface marqueur : `IASTdeclaration`;
- nœuds `IASTassignment` et `IASTfunctionDefinition`;
- les implantations `ASTassignment` et `ASTfunctionDefinition` sont de simples conteneurs; (attributs constants + constructeur + getters)
- le nom d'une fonction est encapsulé dans une `IASTvariable`.
variables et fonctions partagent les même problèmes de portée et de renommage

IASTprogram.java

```
public interface IASTprogram
extends com.paracampus.ilp1.interfaces.IASTprogram {
    IASTfunctionDefinition[] getFunctionDefinitions();
}
```

ASTprogram.java

```
public class ASTprogram extends com.paracampus.ilp1.ast.ASTprogram
implements IASTprogram {

    public ASTprogram(IASTfunctionDefinition[] functions, IASTexpression expression)
    {
        super(expression);
        this.functions = Arrays.asList(functions);
    }

    protected List<IASTfunctionDefinition> functions;

    @Override public IASTfunctionDefinition[] getFunctionDefinitions()
    { return functions.toArray(new IASTfunctionDefinition[0]); }
}
```

- un programme maintient la liste des **ses définitions de fonctions**.

Extension de l'analyse syntaxique : *Listener*

ILPMLListener.java (ILP2)

```
@Override public void exitVariableAssign(VariableAssignContext ctx) {
    ctx.node = factory.newAssignment(
        factory.newVariable(ctx.var.getText()),
        ctx.val.node);
}

@Override public void exitGlobalFunDef(GlobalFunDefContext ctx) {
    ctx.node = factory.newFunctionDefinition(
        factory.newVariable(ctx.name.getText()),
        toVariables(ctx.vars, false),
        ctx.body.node);
}

@Override public void exitProg(ProgContext ctx) {
    List<IASTfunctionDefinition> f = new ArrayList<>();
    for (GlobalFunDefContext d : ctx.defs)
        f.add((IASTfunctionDefinition)(d.node));
    ctx.node = factory.newProgram(
        f.toArray(new IASTfunctionDefinition[0]),
        factory.newSequence(toExpressions(ctx.exprs)));
}
```

ANTLR se charge de placer toutes les définitions de fonctions dans `ctx.defs` et les expressions *top-level* dans `ctx.exprs`.

⇒ se sont les attributs d'`ASTprogram` !

Environnements d'interprétation : rappels (1/2)

ILexicalEnvironment (ILP1)

```
public interface ILexicalEnvironment
extends IEnvironment<IASTvariable, Object, EvaluationException>
{
    @Override ILexicalEnvironment extend(IASTvariable variable, Object value);
    @Override ILexicalEnvironment getNext() throws EvaluationException;
    // inherited from IEnvironment
    Object getValue(IASTvariable key) throws EvaluationException;
    void update(IASTvariable key, Object value) throws EvaluationException;
}
```

Environnement local : (lexical)

- maintient la valeur des **variables locales**; (bloc **let**)
- l'environnement est passé en argument au visiteur;
⇒ un nœud peut évaluer un sous-nœud dans un environnement différent
- **let x = e1 in e2 empile** une association pour **x** pendant l'évaluation de **e2**;
- en sortie de bloc **e2**, l'environnement est **restauré** : **x** est dépilé;
- plusieurs variables de même nom peuvent coexister dans l'environnement, une lecture **trouvera l'association ajoutée le plus récemment**.

Environnements d'interprétation : rappels (2/2)

```
IGlobalVariableEnvironment (ILP1)

public interface IGlobalVariableEnvironment
{
    Object getGlobalVariableValue (String variableName);
    void addGlobalVariableValue (String variableName, Object value);
    void addGlobalVariableValue (IPrimitive primitive);
    void updateGlobalVariableValue (String variableName, Object value);
}
```

Environnement global :

- un **environnement global** de l'interprète
maintenu dans un attribut de l'interprète ; (cf. `getGlobalVariableEnvironment()`)
- initialisé **avant** l'interprétation :
 - avec les primitives : variables et fonctions (`pi`, `print`, ...);
 - et les **fonctions globale** définies par l'utilisateur ;
- utilisé et mis à jour **pendant** l'interprétation :
 - affectation : **ajout** ou **modification** d'une **variable globale** ;
 - **lecture** : de la valeur d'une **variable globale**, fonction ou primitive.

Interprétation des lectures et des affectations

Interpreter.java (rappel ILP1)

```
@Override public Object visit(IASTvariable iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    try { return lexenv.getValue(iast); }
    catch (EvaluationException exc) {
        { return getGlobalVariableEnvironment().getGlobalVariableValue(iast.getName()); }
    }
}
```

Interpreter.java (version ILP2)

```
@Override public Object visit(IASTassignment iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    IASTvariable var = iast.getVariable();
    Object val = iast.getExpression().accept(this, lexenv);
    try { lexenv.update(var, val); }
    catch (EvaluationException exc) {
        { getGlobalVariableEnvironment().updateGlobalVariableValue(var.getName(), val); }
        return val;
    }
}
```

- recherche en **priorité** dans l'**environnement local** ;
- si pas de locale trouvée, recherche dans l'**environnement global**
- si toujours non trouvée, l'affectation **ajoute** une variable globale, mais une lecture retourne une erreur ;
- **retour** de la valeur lue ou affectée.

Interprétation des déclarations de fonctions

Interpreter.java (ILP2 simplifié)

```
public Object visit(IASTprogram iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    for ( IASTfunctionDefinition fd : iast.getFunctionDefinitions() ) {
        Object f = this.visit(fd, lexenv);
        getGlobalVariableEnvironment().addGlobalVariableValue(fd.getName(), f);
    }
    try { return iast.getBody().accept(this, lexenv); }
    catch (Exception exc) { return exc; }
}

public Invocable visit(IASTfunctionDefinition iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    return new Function(iast.getVariables(), iast.getBody(), new EmptyLexicalEnvironment());
}
```

- **extraction** des déclarations de fonctions de l'AST ;
- et enregistrement dans l'**environnement global initial**
sous forme de **Fonction** ; (transparent suivant)
le corps des fonctions n'est pas interprété lors de la définition
l'environnement local correspondra à une clôture... en ILP3 ; pour l'instant, il est vide
- avant de lancer l'interprète sur les expressions du programme.
le corps des fonctions est interprété lors des appels à la fonction

Interprétation des invocations de fonctions (1/2)

Invocable.java (rappel ILP1)

```
public interface Invocable {  
    int getArity();  
    Object apply(Interpreter interpreter, Object[] argument) throws ...;  
}
```

Function.java (rappel ILP1)

```
public class Function implements IFunction /* = Invocable */ {  
    @Override public int getArity() ...  
    public IASTvariable[] getVariables() ...  
    public IASTexpression getBody() ...  
  
    @Override public Object apply(Interpreter interpreter, Object[] argument) throws ... {  
        if ( argument.length != getArity() ) throw new EvaluationException("Wrong arity");  
        ILexicalEnvironment lexenv2 = getClosedEnvironment();  
        IASTvariable[] variables = getVariables();  
        for ( int i=0 ; i<argument.length ; i++ )  
            lexenv2 = lexenv2.extend(variables[i], argument[i]);  
        return getBody().accept(interpreter, lexenv2);  
    }  
}
```

- **conteneur** pour le corps, l'arité et les arguments formels ;
- méthode **apply** qui lie les arguments formels et réels et appelle récursivement l'interprète sur le corps de la fonction.

l'environnement local ne contient (en ILP2) que les arguments

Interprétation des invocations de fonctions (2/2)

Interpreter.java (ILP2)

```
@Override public Object visit(IASTInvocation iast, ILexicalEnvironment lexenv)
throws EvaluationException {
    Object function = iast.getFunction().accept(this, lexenv);
    if ( function instanceof Invocable ) {
        Invocable f = (Invocable)function;
        List<Object> args = new Vector<Object>();
        for ( IASTExpression arg : iast.getArguments() ) {
            Object value = arg.accept(this, lexenv);
            args.add(value);
        }
        return f.apply(this, args.toArray());
    } else {
        String msg = "Cannot apply " + function;
        throw new EvaluationException(msg);
    }
}
```

- évaluation de l'expression fonction en une valeur pointant vers la fonction ;
ce doit être un `Invocable`
- évaluation des expressions passées en argument ;
- puis appel par `apply` à l'`Invocable`, qui fait le reste ;
- le visiteur d'interprétation `this` est passé en argument à `apply`,
pour qu'il puisse évaluer (récurivement) le corps de la fonction.

Difficultés spécifiques à la compilation :

1. mettre à jour la **normalisation** ;

nouvelles formes de variables et d'appels de fonctions

2. **déclaration** des variables et fonctions :

- en ILP : elles peuvent être référencées avant d'être déclarées ;
- en C : déclaration obligatoire avant toute référence ;

le C distingue déclaration et définition

⇒ il faut parcourir l'AST pour collecter toutes les fonctions
générer la liste des déclarations de fonction C
puis générer la définition des fonctions C avec leur corps ;

3. stockage possible de **fonctions dans des variables** qui seront appelées ultérieurement.

influence sur la représentation des données `ILP_Object`

Compilateur : invocation directe et indirecte de fonctions

appel direct (ILP)

```
function double(x) (2 * x);  
double(27)
```

appel indirect (ILP)

```
function double(x) (2 * x);  
let f = double in f(3) - 8
```

appel direct (C généré)

```
ILP_Object ilp_double  
(ILP_Closure ilp_useless, ILP_Object x1)  
{  
    ILP_Object ilptmp2267;  
    ilptmp2267 = ILP_Integer2ILP (2);  
    return ILP_Times (ilptmp2267, x1);  
}  
  
ILP_Object ilp_program ()  
{  
    return ilp_double (  
        NULL,  
        ILP_Integer2ILP (27));  
}
```

appel indirect (C généré)

```
struct ILP_Closure double_closure_object = {  
    &ILP_object_Closure_class, {{ilp_double, 1, {NULL}}}  
};  
  
ILP_Object ilp_program ()  
{  
    ILP_Object f2 = &double_closure_object;  
    {  
        ILP_Object ilptmp2412 =  
            ILP_invoke (f2, 1, ILP_Integer2ILP(3));  
        return ILP_Minus (ilptmp2412, ILP_Integer2ILP(8));  
    }  
}
```

Difficulté : appeler une fonction référencée par une variable.

Bibliothèque d'exécution : invocations indirectes (1/2)

ilp.h

```
typedef struct ILP_Object* (*ILP_general_function)();

typedef struct ILP_Closure {
    struct ILP_Class* _class;
    union {
        struct asClosure_ {
            ILP_general_function function;
            short arity;
            struct ILP_Object* closed_variables[1];
        } asClosure;
    } _content;
} *ILP_Closure;
```

exemple

```
ILP_Object ilp_double (ILP_Closure ilp_useless, ILP_Object x1);

struct ILP_Closure double_closure_object =
    { &ILP_object_Closure_class, {{ilp_double, 1, {NULL}}}} };
```

- **function** est un pointeur de fonction C générique (`ILP_Object (*)()`);
- l'arité est spécifiée dans un autre champ de la clôture : **ariy**;
- **closed_variables** ne servira que pour les fonctions de première classe, en ILP3; en ILP2, ce sera toujours un tableau de taille 1, contenant **NULL** (liste vide).

Bibliothèque d'exécution : invocations indirectes (2/2)

ilp.c

```
ILP_Object ILP_invoke (ILP_Object closure, int argc, ...) {
    va_list args;
    ILP_Object result;
    ILP_general_function f = ILP_find_invokee(closure, argc);
    va_start(args, argc);
    switch ( argc ) {
        case 0: {
            result = f(closure);
            break;
        }
        case 1: {
            ILP_Object arg1 = va_arg(args, ILP_Object);
            result = f(closure, arg1);
            break;
        }
        case 2: {
            ILP_Object arg1 = va_arg(args, ILP_Object);
            ILP_Object arg2 = va_arg(args, ILP_Object);
            result = f(closure, arg1, arg2);
            break;
        }
        ...
    }
    va_end(args);
    return result;
}
```

`ILP_find_invokee` (non montrée) vérifie que `closure` est de type fonction, et son arité est `argc`.

AST normalisé (ASTC) : rappels

Le compilateur commence par transformer l'AST en ASTC.

Toujours les mêmes deux buts, avec quelques nouveautés pour ILP2 :

Classification :

- distinction du type de portée :
 - variable locale : `ASTClocalVariable` ;
 - variable globale : `ASTCglobalVariable` ;
 - fonction globale : `IASTCglobalFunctionVariable` ;

il faut distinguer une fonction, d'une variable globale contenant une fonction

e.g., dans `let f = double`, `f` est une `IASTCglobalVariable`
`double` est une `IASTCglobalFunctionVariable`

- distinction du type d'appel :
 - direct, par nom : `ASTCglobalInvocation` ;
 - indirect, par variable : `ASTCcomputedInvocation`.

Partage et identification :

- chaque `ASTCvariable` identifie de manière unique une variable ;
- toutes les utilisations de la même variable partagent le même nœud.

IASTCfunctionDefinition.java

```
public interface IASTCfunctionDefinition
extends IASTfunctionDefinition, IClosable {
    String getName();
}
```

IASTCprogram.java (ILP2 simplifié)

```
public interface IASTCprogram
extends com.paracampus.ilp1.compiler.interfaces.IASTCprogram {
    @Override Set<IASTCglobalVariable> getGlobalVariables();
    @Override void setGlobalVariables(Set<IASTCglobalVariable> gvs);
    IASTCfunctionDefinition[] getFunctionDefinitions();
}
```

- **IASTCfunctionDefinition** : ajout du **nom C** ;
ainsi qu'une notion de clôture, **IClosable**, mais cela ne servira qu'en ILP3...
- **IASTCglobalVariable** et **IASTCglobalFunctionVariable** :
interfaces **marqueur** ; (pour distinguer les cas avec **instanceof**)
- **getGlobalVariables()** retourne une référence à un ensemble ;
le client peut modifier le contenu de cet ensemble avec **add** et **remove**.

Nouvelles classes d'AST normalisé

ASTCfunctionDefinition.java

```
public class ASTCfunctionDefinition extends ASTfunctionDefinition
implements IASTCfunctionDefinition {
    public String getName()
    { return "ilp_" + getFunctionVariable().getMangledName(); }
}
```

ASTCprogram.java (ILP2)

```
public class ASTCprogram
extends com.paracampus.ilp1.compiler.ast.ASTCprogram
implements com.paracampus.ilp2.compiler.interfaces.IASTCprogram {
    protected List<IASTfunctionDefinition> functions;

    public ASTCprogram (IASTCfunctionDefinition[] functions, IASTexpression expression)
    { super(expression); this.functions = functions; }

    protected IASTCfunctionDefinition[] functions;

    @Override public IASTCfunctionDefinition[] getFunctionDefinitions()
    { return functions; }
}
```

- `getName` ajoute un préfixe `ilp_` ;
- `ASTCprogram` ajoute un conteneur de fonctions, fixé à la construction ;
- à mettre à jour par héritage : `(I)NormalizationFactory`.

Normalisation de l'AST (1/4)

Normalizer.java (ILP2)

```
public IASTCprogram transform(IASTprogram program) throws CompilationException
{
    INormalizationEnvironment env = NormalizationEnvironment.EMPTY;
    IASTfunctionDefinition[] functions = program.getFunctionDefinitions();
    IASTCfunctionDefinition[] funs = new IASTCfunctionDefinition[functions.length];
    for ( IASTfunctionDefinition function : functions ) {
        IASTCglobalFunctionVariable gfv =
            ((INormalizationFactory)factory).newGlobalFunctionVariable(function.getName());
        env = env.extend(gfv, gfv);
    }
    for ( int i=0 ; i<functions.length ; i++ ) {
        IASTfunctionDefinition function = functions[i];
        IASTCfunctionDefinition newfunction = visit(function, env);
        funs[i] = newfunction;
    }
    IASTexpression body = program.getBody();
    IASTexpression newbody = body.accept(this, env);
    return ((INormalizationFactory)factory).newProgram(funs, newbody);
}
```

Point d'entrée **IASTprogram** :

- ajout des variables globales correspondant aux fonctions;
- transformation des fonctions;
- transformation des expressions hors fonctions.

```
public IASTCfunctionDefinition visit(IASTfunctionDefinition iast, INormalizationEnvironment env)
throws CompilationException {
    String functionName = iast.getName();
    IASTvariable[] variables = iast.getVariables();
    IASTvariable[] newvariables = new IASTvariable[variables.length];
    INormalizationEnvironment newenv = env;
    for ( int i=0 ; i<variables.length ; i++ ) {
        IASTvariable variable = variables[i];
        IASTvariable newvariable = factory.newLocalVariable(variable.getName());
        newvariables[i] = newvariable;
        newenv = newenv.extend(variable, newvariable);
    }
    IASTexpression newbody = iast.getBody().accept(this, newenv);
    IASTvariable functionVariable =
        ((INormalizationFactory)factory).newGlobalFunctionVariable(functionName);
    return ((INormalizationFactory)factory).newFunctionDefinition(
        functionVariable, newvariables, newbody);
}
```

Transformation d'une fonction `IASTfunctionDefinition` :

- enregistrement des arguments formels comme variables locales ;
- visite du corps ;
- création d'une variable globale de fonction.

Normalisation de l'AST (3/4)

Normalizer.java (ILP2)

```
@Override public IASTExpression visit(IASTInvocation iast, INormalizationEnvironment env)
throws CompilationException {
    IASTExpression funexpr = iast.getFunction().accept(this, env);
    IASTExpression[] arguments = iast.getArguments();
    IASTExpression[] args = new IASTExpression[arguments.length];
    for ( int i=0 ; i<arguments.length ; i++ ) {
        IASTExpression argument = arguments[i];
        IASTExpression arg = argument.accept(this, env);
        args[i] = arg;
    }
    if ( funexpr instanceof IASTCglobalVariable ) {
        IASTCglobalVariable f = (IASTCglobalVariable) funexpr;
        return ((INormalizationFactory)factory).newGlobalInvocation(f, args);
    }
    else return ((INormalizationFactory)factory).newComputedInvocation(funexpr, args);
}
```

Transformation d'un appel `IASTInvocation` :

- transformation de la référence à la fonction ;
- transformation des arguments ;
- création d'un nœud ASTC selon le type de référence de fonction `newGlobalInvocation` ou `newComputedInvocation`.

en réalité, c'est une optimisation ; `newComputedInvocation` pourrait être utilisé partout

Normalizer.java (rappel ILP1)

```
@Override public IASTvariable visit(IASTvariable iast, INormalizationEnvironment env)
throws CompilationException
{
    try {
        return env.renaming(iast);
    } catch (NoSuchLocalVariableException exc) {
        for ( IASTvariable gv : globalVariables ) {
            if ( iast.getName().equals(gv.getName()) ) {
                return gv;
            }
        }
        IASTvariable gv = factory.newGlobalVariable(iast.getName());
        globalVariables.add(gv);
        return gv;
    }
}
```

Transformation d'une variable **IASTvariable**, héritée d'ILP1 :

- recherche la variable en priorité dans l'environnement local ;
- si non trouvée, cherche une variable globale ;
- si aucune variable n'est trouvée, une variable globale est créée.

Déclaration des variables et fonctions globales

source ILP

```
function f(x)
  let y = x +1 in
  g = x
```

code C généré (simplifié)

```
/* Global variables */
ILP_Object g;

/* Global prototypes */
ILP_Object ilp__f (ILP_Closure ilp_useless, ILP_Object x1);

/* Global functions */
ILP_Object ilp__f (ILP_Closure ilp_useless, ILP_Object x1)
{
  ILP_Object y2 = ILP_Plus (x1, ILP_Integer2ILP (1));
  return (g = x1);
}

struct ILP_Closure f_closure_object =
{ &ILP_object_Closure_class, {{ilp__f, 1, {NULL}}}} ;
```

- déclaration en avance des fonctions et des variables de fonctions ;
nécessaire pour les appels récursifs
⇒ facile grâce à `getFunctionDefinitions` dans `ASTCprogram` ;
- déclaration en avance des variables globales
⇒ nécessite un **parcours complet de l'AST** pour les retrouver toutes !
attention : les variables globales ne sont pas automatiquement initialisées !

$$expr ::= 1 \mid glob \mid loc \mid expr + expr \mid \text{let } loc = expr \text{ in } expr$$

Illustration sur un langage simple d'expressions $expr$.

Les variables sont **déjà classifiées** en variables locales et variables globales
 $\Rightarrow$ il suffit de collecter les globales uniquement.

Variables globales de $expr$: $GV(expr) \subseteq variables$

Défini par **récurrence sur la syntaxe** : (un nœud est défini en fonction de ses sous-nœuds)

- $GV(1) = \emptyset$
- $GV(loc) = \emptyset$
- $GV(glob) = \{glob\}$
- $GV(e_1 + e_2) = GV(e_1) \cup GV(e_2)$
- $GV(\text{let } var = e_1 \text{ in } e_2) = GV(e_1) \cup GV(e_2)$

$\Rightarrow$ on reconnaît un motif de **visiteur** retournant un **ensemble de variables**.

Implantation : GlobalVariableCollector (1/2)

GlobalVariableCollector.java (ILP1, fragment)

```
public class GlobalVariableCollector
implements IASTCvisitor<Set<IASTCglobalVariable>, Set<IASTCglobalVariable>, ...> {

    protected Set<IASTCglobalVariable> result;
    public GlobalVariableCollector () { this.result = new HashSet<>(); }

    public Set<IASTCglobalVariable> analyze(IASTCprogram program) throws ... {
    { result = program.getBody().accept(this, result); return result; }

    @Override public Set<IASTCglobalVariable>
    visit(IASTvariable iast, Set<IASTCglobalVariable> result) throws ... {
        if ( iast instanceof IASTCglobalVariable )
            result = visit((IASTCglobalVariable) iast, result);
        return result;
    }

    @Override public Set<IASTCglobalVariable>
    visit(IASTCglobalVariable iast, Set<IASTCglobalVariable> result) throws ... {
    { result.add(iast); return result; }

    @Override public Set<IASTCglobalVariable>
    visit(IASTClocalVariable iast, Set<IASTCglobalVariable> result) throws ...
    { return result; }
```

Optimisation : plutôt que de créer un nouvel ensemble à chaque appel, nous utilisons **un seul ensemble**, qui est **enrichi** durant le parcours puis retourné.

Implantation : GlobalVariableCollector (2/2)

GlobalVariableCollector.java (ILP1, fragment)

```
@Override public Set<IASTCglobalVariable>
visit(IASTbinaryOperation iast, Set<IASTCglobalVariable> result) throws ... {
    iast.getLeftOperand().accept(this, result);
    iast.getRightOperand().accept(this, result);
    return result;
}

@Override public Set<IASTCglobalVariable>
visit(IASTinteger iast, Set<IASTCglobalVariable> result) throws ...
{ return result; }
...
```

GlobalVariableCollector.java (ILP2, fragment)

```
public class GlobalVariableCollector
extends com.paracampus.ilp1.compiler.GlobalVariableCollector
implements IASTCvisitor<Set<IASTCglobalVariable>, Set<IASTCglobalVariable>, ...> {

    public Set<IASTCglobalVariable> analyze(IASTCprogram program) throws ... {
        for ( IASTfunctionDefinition ifd : program.getFunctionDefinitions() ) {
            result = ifd.getBody().accept(this, result);
        }
        result = program.getBody().accept(this, result);
        return result;
    }
    ...
}
```


Compiler.java (ILP2, fragment)

```
public String compile(IASTprogram program) throws CompilationException
{
    IASTCprogram newprogram = normalize(program);
    newprogram = ((IASTCprogram) optimizer.transform(newprogram));

    GlobalVariableCollector gvc = new GlobalVariableCollector();
    Set<IASTCglobalVariable> gvs = gvc.analyze(newprogram);
    newprogram.setGlobalVariables(gvs);

    // FreeVariableCollector omis

    Context context = new Context(NoDestination.NO_DESTINATION);
    ...
    visit(newprogram, context);
    ...
}
```

Collecte des variables globales en sortie de normalisation (`analyze`)
et mise à disposition au reste du visiteur (`setGlobalVariables`).

Compilateur : point d'entrée (2/2)

Compiler.java (ILP2, fragment)

```
public Void visit(IASTCprogram iast, Context context) throws CompilationException
{
    emit(cProgramPrefix); // en-têtes

    emit(cGlobalVariablesPrefix); // commentaire
    for ( IASTCglobalVariable gv : iast.getGlobalVariables() ) {
        emit("ILP_Object ");
        emit(gv.getMangledName());
        emit(";\n");
    }
    emit(cGlobalVariablesSuffix); //saut de ligne

    emit(cPrototypesPrefix); // commentaire
    Context c = context.redirect(NoDestination.NO_DESTINATION);
    for ( IASTfunctionDefinition ifd : iast.getFunctionDefinitions() )
        this.emitPrototype(ifd, c);

    emit(cFunctionsPrefix); // commentaire
    for ( IASTfunctionDefinition ifd : iast.getFunctionDefinitions() ) {
        this.visit(ifd, c);
        emitClosure(ifd, c);
    }
    emit(cFunctionsSuffix); // saut de ligne

    // génération du corps, comme en ILP1
}
```

Schéma de compilation : IASTCglobalFunctionVariable

 $\overrightarrow{var}^d$

d (ILP_Object) &var_closure_object;

IASTClocalVariable

 $\overrightarrow{var}^d$

d var.getMangledName();

IASTCglobalVariable

 $\overrightarrow{var}^d$

d globalVariableEnvironment.getCName(var);

Dans `visit(IASTvariable iast, Context context)`, réorientation selon le type `T` de la variable : `if (iast instanceof T) visit((T)iast, context)` pour

- `T = IASTClocalVariable;`
- `T = IASTCglobalVariable;`
- `T = IASTCglobalFunctionVariable.`

Schéma de compilation (simplifié)

```
var = exprd  
-----  
{  
    ILP_Object tmp;  
    expr(tmp=)  
    d (var = tmp);  
}
```

Selon que *var* est locale ou globale, on utilisera en fait soit `var.getMangledName`, soit `getCName(var)`.

Utilisation du fait qu'une affectation est une expression valide en C.

On peut écrire : `return (x = 2)` ou `tmp = (x = 2)`.

Si *var* est locale et close, alors elle est emboîtée dans une structure `ILP_Box`, ce qui rend l'affectation plus complexe... mais ce n'est pas encore possible en ILP2 ; nous verrons cela avec les fonctions de première classe, en ILP3.

Schéma de compilation : définition

```


$$\frac{}{\text{function } name(arg1, \dots, argN) \xrightarrow{d} expr}$$



---


ILP_Object ilp__name(ILP_Closure ilp_useless,
                    ILP_Object arg1, \dots, ILP_Object argN)
{
     $\xrightarrow{expr} (return)$ 
}
```

Prototype et clôture

```

ILP_Object ilp__name(ILP_Closure ilp_useless,
                    ILP_Object arg1, \dots, ILP_Object argN);

struct ILP_Closure name_closure_object = {
    &ILP_object_Closure_class,
    {{ ilp__name, N, {NULL}}}
};
```

Schéma de compilation : cas IASTCglobalFunctionVariable

```

                                 $\xrightarrow{\text{d}}$ 
                                 $\text{var}(\text{arg1}, \dots, \text{argN})$ 


---


{
    ILP_Object tmp1;
    ...
    ILP_Object tmpN;
     $\xrightarrow{\text{d}}$ 
     $\text{arg1}(\text{tmp1=})$ 
    ...
     $\xrightarrow{\text{d}}$ 
     $\text{argN}(\text{tmpN=})$ 
    d ilp__var.getMangledName() (tmp1, ..., tmp2);
}
```

Le nom de la fonction appelée est **connu statiquement**.

On génère un appel direct à la fonction C correspondante.

Schéma de compilation : cas général

```


$$\frac{}{\overrightarrow{\text{expr}(\text{arg1}, \dots, \text{argN})}^d}$$



---


{
    ILP_Object tmpF;
    ILP_Object tmp1;
    ...
    ILP_Object tmpN;
     $\overrightarrow{\text{expr}}^{(tmpF=)}$ 
     $\overrightarrow{\text{arg1}}^{(tmp1=)}$ 
    ...
     $\overrightarrow{\text{argN}}^{(tmpN=)}$ 
    d ILP_invoke(tmpF, tmp1, ..., tmpN);
}
```

Le nom de la fonction appelée n'est pas connu statiquement.

Il est nécessaire d'évaluer une expression à l'exécution pour trouver la fonction, puis de l'appeler par pointeur avec `ILP_invoke`

Les points essentiels vus (ou revus) :

- étapes principales d'une **extension d'ILP** ;
- **désambiguation** et **résolution** des variables ;
- **catégorisation** des **accès aux variables** et des **appels de fonction** ;
analyse statique
- **aspects statiques, aspects dynamiques** ;
- **visiteurs** pour :
 - interpréter ;
 - transformer ($\text{AST} \rightarrow \text{ASTC} \rightarrow \text{code C}$) ;
 - extraire des informations (collecte des variables) ;
- **environnements** : pour l'exécution, pour la transformation ;
- **schémas de compilation**.
à voir chez soi : le générateur de code correspondant aux schémas, dans `Compiler.java`